

Engenharia de Computação
Inteligência Artificial
12 - Algoritmos Genéticos
Prática Guiada: Evolução de Imagens Binárias

Objetivo da prática - 55 minutos

Retomando

Na aula teórica, estudamos população, fitness, seleção, recombinação, mutação e elitismo. Agora vamos conectá-los em uma implementação mínima.

Desafio

Evoluir uma imagem binária simples, aproximando uma imagem-alvo de 4×4 pixels. O código será desenvolvido por etapas, não fornecido pronto.

Observação

O problema é didático. Em uma aplicação real, copiar diretamente a imagem seria trivial; aqui o objetivo é compreender o mecanismo do AG.

Roteiro e checkpoints

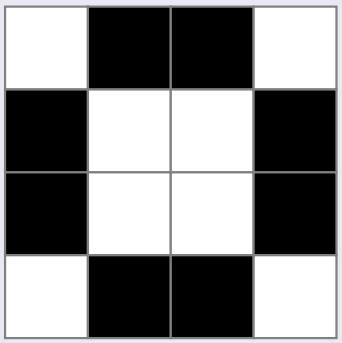
Etapa	Tempo
1. Representação da imagem e população	7 min
2. Função de fitness e testes	8 min
3. Seleção por torneio	8 min
4. Crossover e mutação	15 min
5. Gerações e experimentação	12 min
6. Conexão com o Trabalho 02	5 min

Entrega da aula

Conseguir executar o ciclo completo e justificar cada etapa. Caso não conclua, apresentar quais checkpoints passaram e onde o código parou.

O desafio visual

Imagem-alvo: 4 por 4



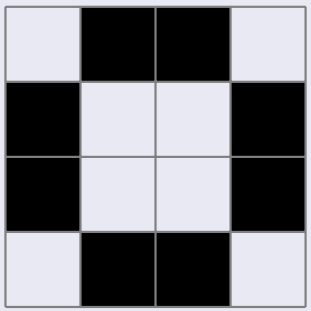
Representação

```
0110
1001
1001
0110
```

- 0 = branco; 1 = preto.
- Um indivíduo contém 16 genes.
- O fitness é a quantidade de posições iguais ao alvo.

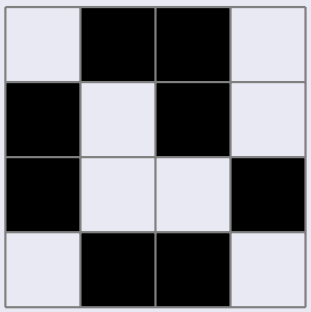
Visualizando a evolução: alvo e candidato

Imagem-alvo



`0110 / 1001 / 1001 / 0110`

Exemplo de candidato



`0110 / 1010 / 1001 / 0110`

Verificação

O candidato difere em 2 pixels; portanto, tem fitness 14 de 16.

Pergunta

Como transformar uma lista de 16 bits em uma matriz 4×4 e visualizá-la com `imshow`?

Checkpoint 1: representar candidatos (7 min)

Implemente

1. Defina o alvo como lista de 16 inteiros (0 ou 1).
2. Escreva `criar_individuo()` para sortear 16 bits.
3. Escreva `criar_populacao(n)` para criar *n* indivíduos.

Interfaces previstas

```
criar_individuo() -> list[int]
criar_populacao(n:  int) -> list[list[int]]
```

Autoverificação

A população tem *n* indivíduos? Todos têm tamanho 16? Todos os genes pertencem a $\{0, 1\}$?

Checkpoint 1: testes mínimos

A propriedade importa mais que os valores sorteados

```
pop = criar_populacao(10)
assert len(pop) == 10
assert all(len(ind) == 16 for ind in pop)
assert all(bit in (0, 1) for ind in pop for bit in ind)
```

Observação

O teste não exige que dois indivíduos sejam diferentes. A inicialização aleatória pode produzir repetições.

Checkpoint 2: como avaliar? (8 min)

Implemente a função de fitness

Para alvo t e candidato x, conte as posições iguais:

$$f(x) = \sum_{i=1}^{16} \mathbf{1}[x_i = t_i].$$

Assinatura: `fitness(individuo, alvo) -> int`.

Pior caso

Nenhum bit coincide: fitness 0.

Melhor caso

Todos os 16 bits coincidem: fitness 16.

Checkpoint 2: testes orientados por propriedades

Antes de evoluir, teste a avaliação

```
assert fitness(alvo, alvo) == 16
invertida = [1 - bit for bit in alvo]
assert fitness(invertida, alvo) == 0
assert 0 <= fitness(criar_individuo(), alvo) <= 16
```

Pergunta

Se a função de fitness estiver errada, a evolução pode parecer funcionar? Como perceber?

Checkpoint 3: seleção por torneio (8 min)

Desenvolva sem biblioteca de AG

1. Sorteie dois indivíduos da população.
2. Compare seus valores de fitness.
3. Retorne o candidato de maior fitness.

Interface

```
selecionar_pai(populacao, alvo) -> list[int]
```

Observação

É possível sortear com reposição: os mesmos candidatos podem participar mais de uma vez.

Checkpoint 3: interpretação

Experimento mental

Uma população possui fitness $[3, 7, 11, 15]$. Em um torneio entre os indivíduos com fitness 7 e 11, quem vence?

Resposta para conferência

Vence o indivíduo de fitness 11. Entretanto, ele não precisa ser o melhor da população inteira: o resultado depende dos participantes sorteados.

Checkpoint 4: crossover (8 min)

Implemente recombinação de um ponto

1. Sorteie um ponto de corte entre 1 e 15.
2. Combine o prefixo do primeiro pai com o sufixo do segundo.
3. Faça o segundo filho com os segmentos complementares.

Interface

```
crossover(pai1, pai2) -> (filho1, filho2)
```

Pergunta

Por que os filhos continuam válidos para uma codificação binária, mesmo após a recombinação?

Checkpoint 4: crossover resolvido

Pais, com corte após o quarto gene

```
P1 = 1100|1010
P2 = 0011|0101
```

Descendentes esperados

```
F1 = 11000101
F2 = 00111010
```

Observação

Exemplo com 8 genes para facilitar a leitura. Na implementação da atividade, cada cromossomo possui 16 genes.

Checkpoint 4: mutação (7 min)

Implemente uma mutação por gene

1. Para cada bit, sorteie uma probabilidade independente.
2. Com probabilidade p_m, troque $0 \leftrightarrow 1$.
3. Retorne o indivíduo mutado.

Interface

```
mutar(individuo, p_m=0.05) -> list[int]
```

Pergunta

Com $p_m = 0$, que diversidade pode surgir? Com $p_m = 1$, o que ocorre com todos os bits?

Mutação: interpretar os valores extremos

$p_m = 0$

Nenhum bit sofre mutação; novas combinações dependerão de crossover e seleção.

$p_m = 1$

Todos os bits são invertidos; não equivale a gerar um indivíduo aleatório.

Pergunta

Em 16 genes independentes, qual a quantidade esperada de mutações para $p_m = 0{,}1$?

Conferência

O valor esperado é $16 \times 0{,}1 = 1{,}6$ genes por indivíduo.

Checkpoint 5: implementar o ciclo (8 min)

Monte a geração seguinte

1. Avalie a população e registre o melhor indivíduo.
2. Preserve uma cópia dele (elitismo de 1 indivíduo).
3. Até completar a população: selecione dois pais, recombine e mute.
4. Gere uma nova população do mesmo tamanho.
5. Repita por até 30 gerações ou até alcançar fitness 16.

Observação

Use cópias para que a mutação não altere inadvertidamente um indivíduo da população anterior ou a elite.

Checkpoint 5: pseudocódigo incompleto

Preencha a lógica em Python

```
populacao = criar_populacao(20)
for geracao in range(30):
    # 1. avaliar e guardar o melhor
    # 2. verificar criterio de parada
    # 3. iniciar nova populacao com elite
    # 4. gerar filhos ate completar a populacao
    # 5. atualizar a populacao
```

Pergunta

O que ocorre se a população possui tamanho ímpar e o crossover produz dois filhos por vez?

Checkpoint 5: invariantes da geração

Verifique após cada atualização da população

```
assert len(populacao) == N
assert all(len(ind) == 16 for ind in populacao)
assert all(bit in (0, 1) for ind in populacao for bit in ind)
assert all(0 <= fitness(ind, alvo) <= 16 for ind in populacao)
```

Pergunta

Com elitismo de 1, por que o melhor fitness registrado não deve diminuir entre gerações?

Observação

Para população ímpar, controle quantos filhos são inseridos; a população deve continuar com exatamente *N* indivíduos.

Experimento e registro (4 min)

Compare duas execuções

1. Execute com população 20, 30 gerações e $p_m = 0{,}01$.
2. Repita com $p_m = 0{,}20$.
3. Registre o melhor fitness final e a curva do melhor fitness por geração, se conseguir plotar.

Taxa de mutação	Melhor fitness	Geração de parada
0,01	_____	_____
0,20	_____	_____

Observação

Uma execução por configuração é exploratória e não permite concluir qual taxa é melhor. A comparação rigorosa exige várias sementes.

Experimento: sementes e evidências

Repita sem alterar o problema

1. Use as mesmas 16 posições-alvo em todas as execuções.
2. Repita cada configuração com sementes 0, 1 e 42.
3. Registre melhor fitness, geração de parada e curva de evolução.
4. Compare resultados com cautela: três repetições são apenas exploração inicial, não prova de superioridade estatística.

Observação

Se usar `random` e `numpy.random`, controle ambos os geradores ou empregue uma única fonte de aleatoriedade.

Discussão: o que observar?

Interprete seus resultados

1. O melhor fitness cresceu monotonicamente? O elitismo ajuda a explicar?
2. As duas execuções alcançaram o mesmo resultado?
3. Mutação maior foi sempre melhor?
4. Quais modificações aumentariam a dificuldade do problema?

Observação

Como são apenas 16 bits, o problema é intencionalmente pequeno. A visualização e o aprendizado dos operadores importam mais que a dificuldade da otimização.

Conexão com o Trabalho 02

Representação	Avaliação	Operadores
Aqui: 16 bits. No trabalho: depende do problema.	Aqui: coincidência de pixels. No trabalho: objetivo e restrições.	Aqui: um ponto e inversão de bits. No trabalho: dependem da codificação.

Pergunta

Por que esse crossover não pode ser transferido diretamente para uma permutação do TSP?

Síntese da atividade

1. A representação determina como uma solução é armazenada.
2. O fitness define o que a população tende a preservar.
3. A seleção favorece candidatos promissores, sem garantia de ótimo.
4. Crossover e mutação produzem variação.
5. Elitismo preserva o melhor resultado conhecido.
6. Experimentos dependem de parâmetros e aleatoriedade.

Próximo passo

Transferir esse raciocínio para a representação e a função objetivo do problema atribuído no Trabalho 02, sem reutilizar operadores incompatíveis.

Parte 2: preparação para o Trabalho 02 (45 min)

Modelagem em grupos

1. **10 min:** descrever a entrada, saída e objetivo do problema escolhido.
2. **10 min:** definir o cromossomo e exemplificar dois candidatos válidos.
3. **10 min:** definir a função objetivo e o tratamento de restrições.
4. **10 min:** propor seleção, recombinação e mutação compatíveis com a representação.
5. **5 min:** socializar uma decisão e uma dificuldade técnica.

Observação

Não é necessário programar a Parte 2 nesta aula. Priorize o modelo de solução e os testes de validade. AG, Enxame e Imunes deverão trabalhar sobre a mesma função objetivo e as mesmas instâncias.

Checklist da modelagem do Trabalho 02

Antes de começar a codificação

1. Um indivíduo codifica uma solução completa e interpretável?
2. A função objetivo distingue soluções melhores e piores?
3. Como identificar uma solução inviável e como tratá-la?
4. Recombinação e mutação mantêm a validade dos candidatos?
5. O que registrar para comparar execuções com sementes distintas?

Evidência de aprendizagem

Registro esperado ao final: uma página ou arquivo com entrada e saída, dois cromossomos válidos, função objetivo, restrições, operadores propostos e um teste de validade. Não é necessário entregar código nesta etapa.

Dúvidas



Dúvida é o começo da sabedoria.